МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Информационные технология»

# Лабораторная работа № N
по дисциплине «Введение в информационные технологии» по теме: «Введение в систему контроля версий Git»

Выполнил студент группы:
Номер группы: БВТ2604
ФИО: Пименов Иван Владимирович
Проверил: Павликов А. Е

Москва 2026

**Оглавление**

Введение .....……..3
Выполнение ..... 4-11
Выводы ..... 12

## Цель и задачи работы:

**Цель работы**: познакомиться с принципами работы системы контроля версий Git, понять назначение систем контроля версий и основные понятия Git, научиться создавать локальный репозиторий и выполнять основные операции с ним через командную строку.

В ходе выполнения лабораторной работы необходимо изучить понятия рабочего каталога, индекса, коммита, истории изменений и ветки, а также научиться создавать коммиты, просматривать историю проекта, сравнивать версии файлов, работать с ветками и выполнять отмену внесённых изменений
**Выполнение**:

## Задание 1: Создание локального репозитория

Создаём рабочую папку git_lab и переходим в неё. Инициализируем

Git-репозиторий командой git init. Проверяем состояние через git status: видим, что репозиторий создан, ветка — main, отслеживаемых файлов пока нет

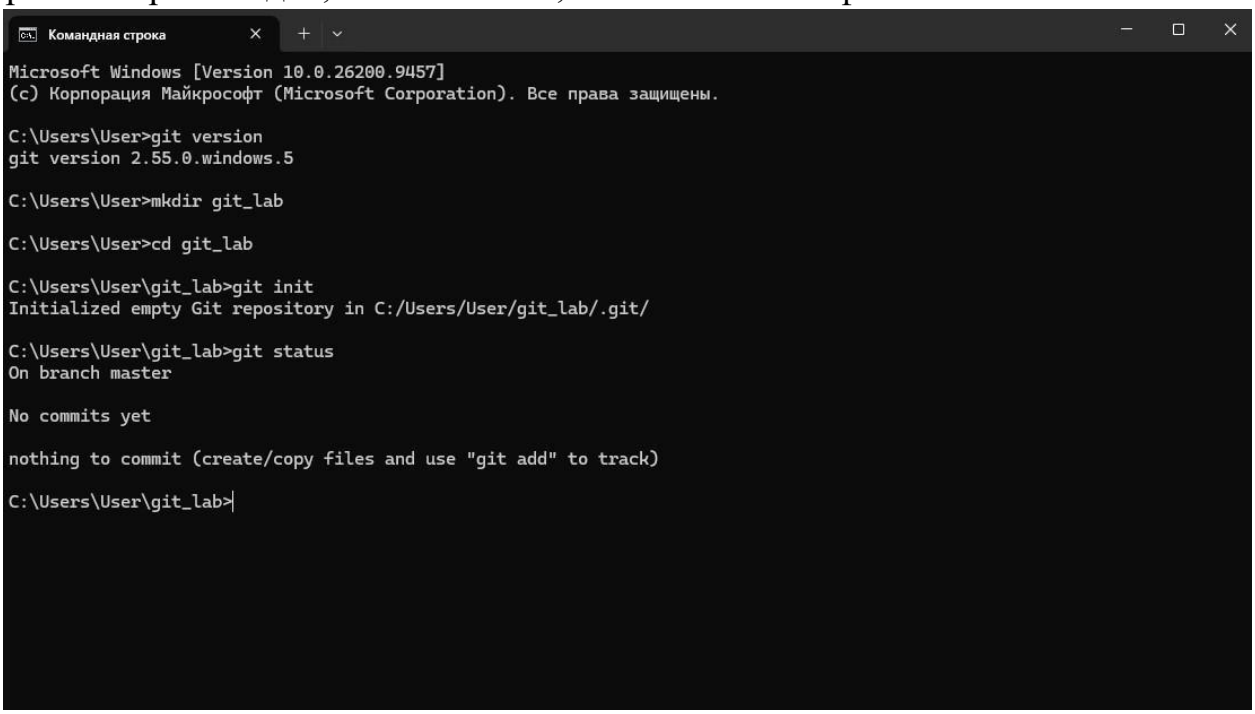



Рисунок 1. Создание локального репозитория

## Задание 2: Первый файл и первый коммит

Создаём файл notes.txt. Сначала добавляем в индекс командой git add, чтобы Git начал отслеживать изменения. Затем делаем первый коммит, сохраняем начальную версию файла в истории. Проверяем историю через git log --oneline: там появляется первая запись.

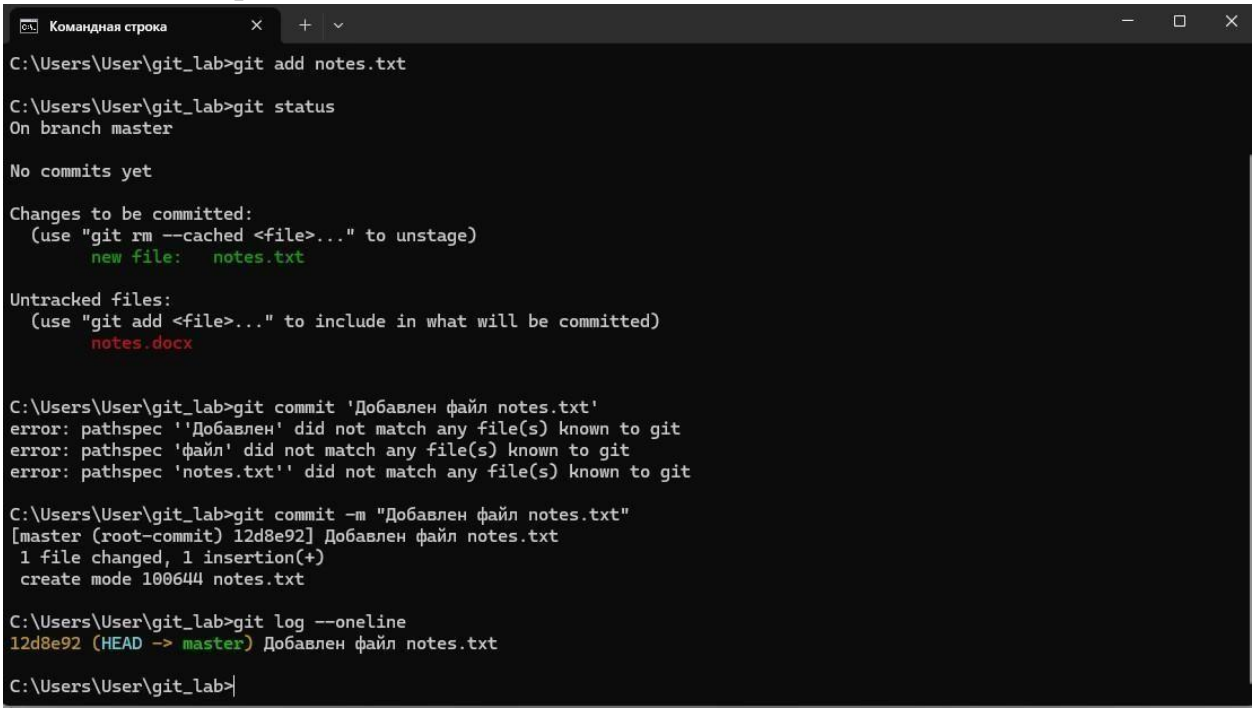



Рисунок 2. Первый коммит

## Задание 3: Изменение файла

Дополняем notes.txt двумя новыми строками. Смотрим разницу в рабочей директории через git diff — это показывает, что именно мы изменили. Добавляем изменения в индекс через git add и снова смотрим разницу, но уже подготовленных к коммиту изменений — для этого используем git diff -staged. Убедившись, что всё верно, фиксируем правки новым коммитом.

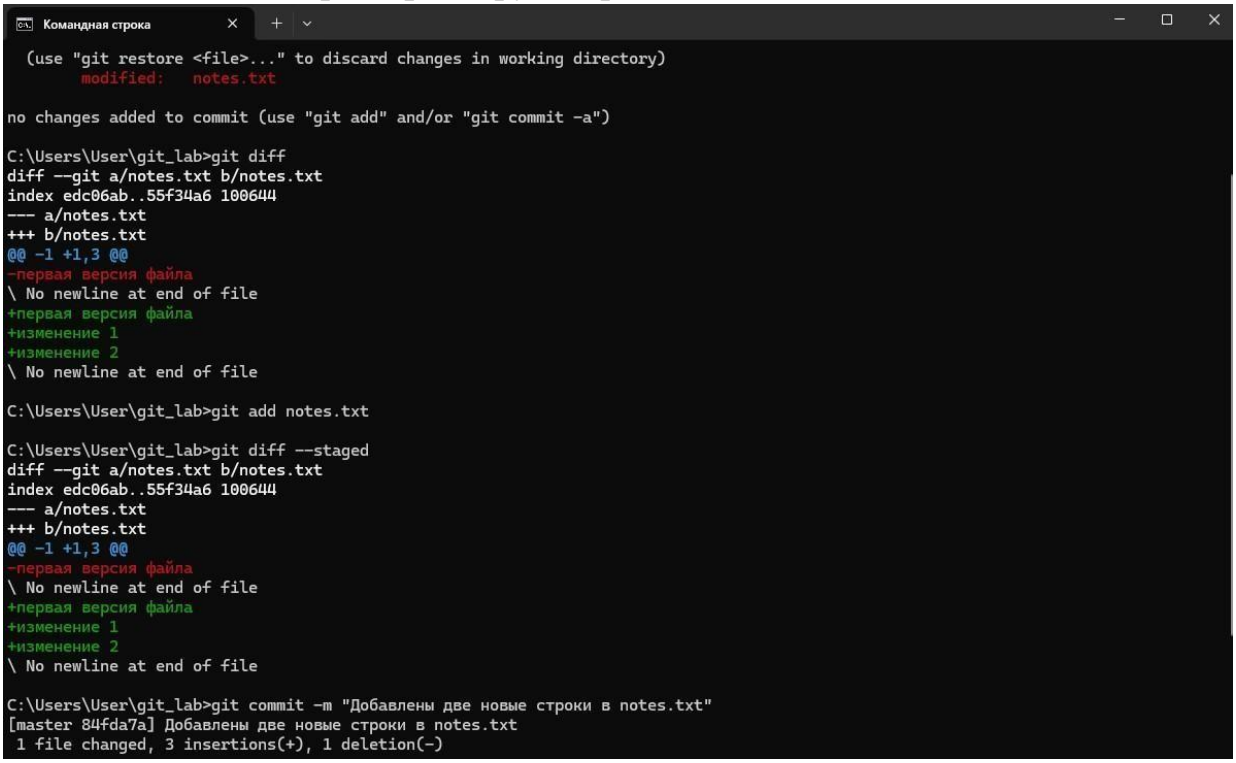



Рисунок 3. Изменение файла

## Задание 4: Создание истории проекта

Вносим ещё три отдельных изменения в notes.txt. Каждое логически завершённое изменение оформляем отдельным коммитом с понятным сообщением. Так мы делаем историю читаемой: по коммитам сразу видно, какую задачу решали на каждом шаге. Проверяем итоговую историю через git log --oneline — там должно быть минимум пять коммитов.

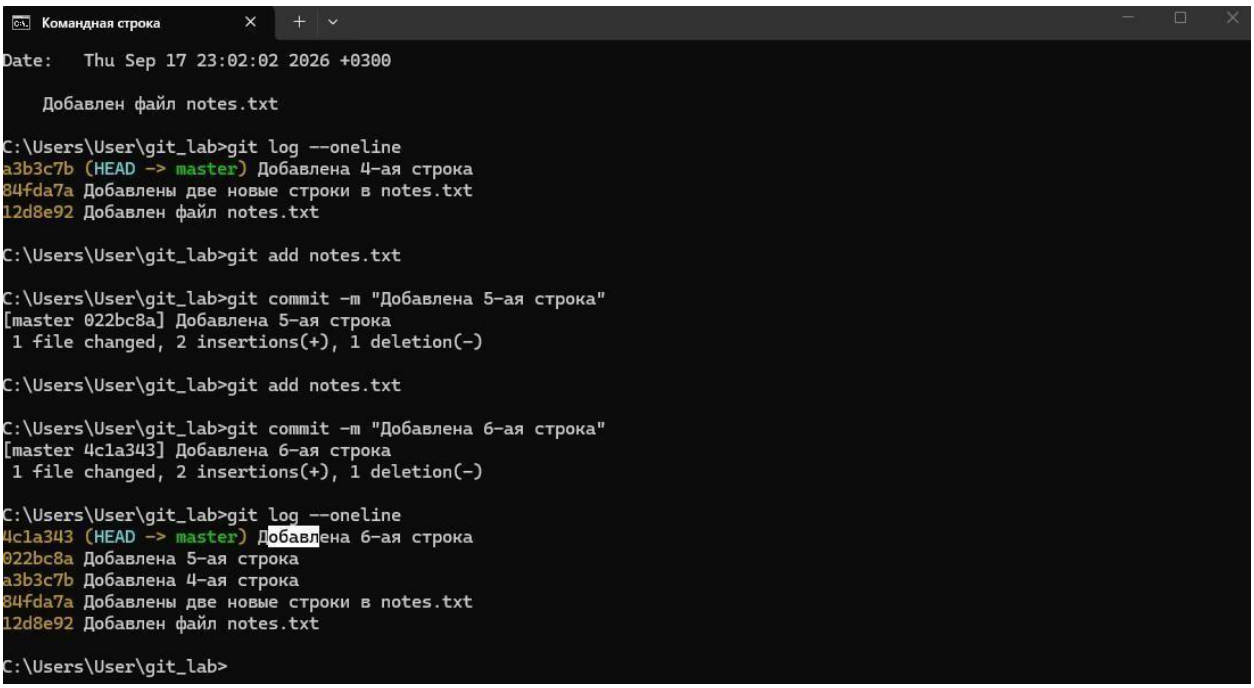



Рисунок 4. Итоговая история проекта

## Задание 5: Работа с веткой

Создаём новую ветку experiment и переключаемся на неё. В этой ветке создаём и коммитим новый файл experiment.txt. Возвращаемся в основную ветку main и проверяем содержимое папки: файла experiment.txt там нет. Это ожидаемо — коммит с файлом существует только в ветке experiment, а ветки работают независимо друг от друга.

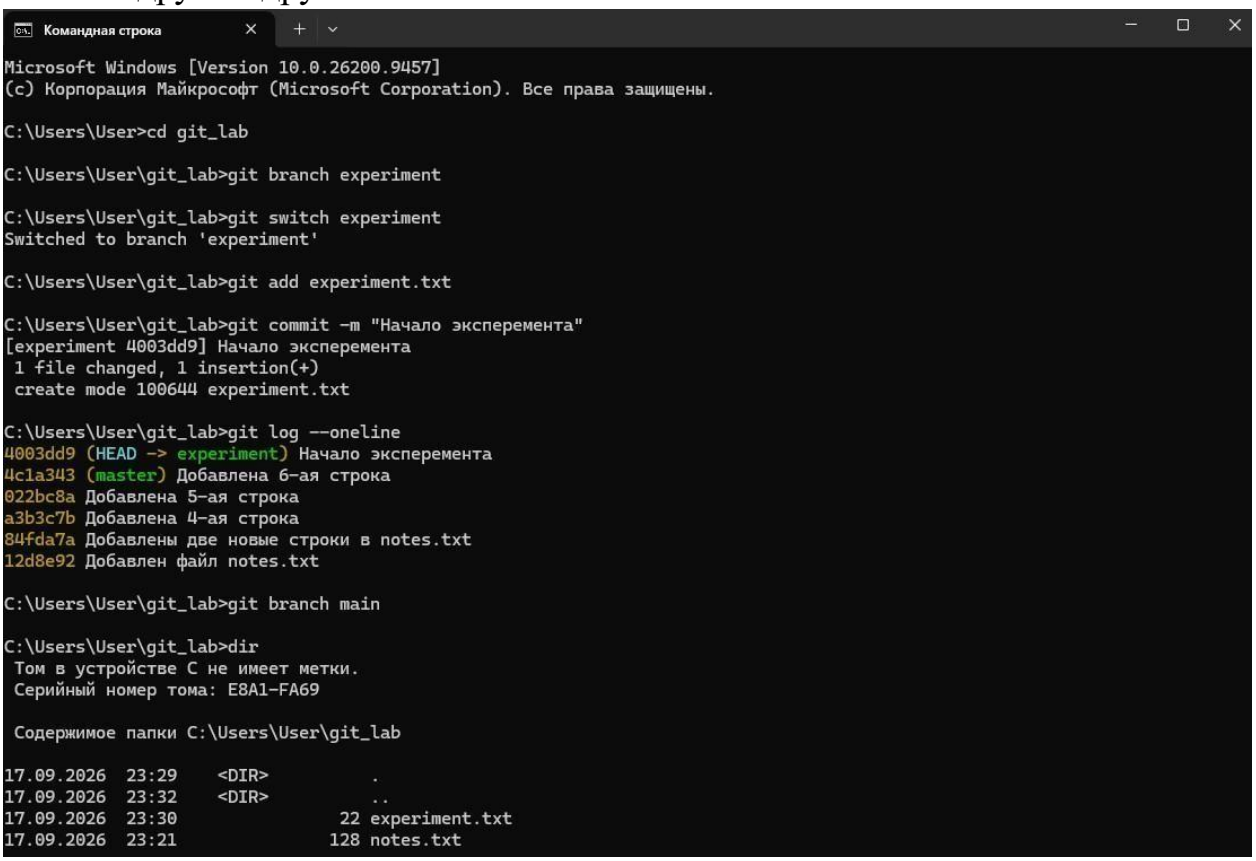



Рисунок 5. Работа с ветками

## Задание 6: Объединение веток

Выполняем слияние ветки experiment в main командой git merge. После этого все изменения из experiment переносятся в main. Проверяем наличие файла experiment.txt — теперь он есть. Смотрим историю через git log --oneline: там виден коммит слияния, который объединяет две линии разработки.

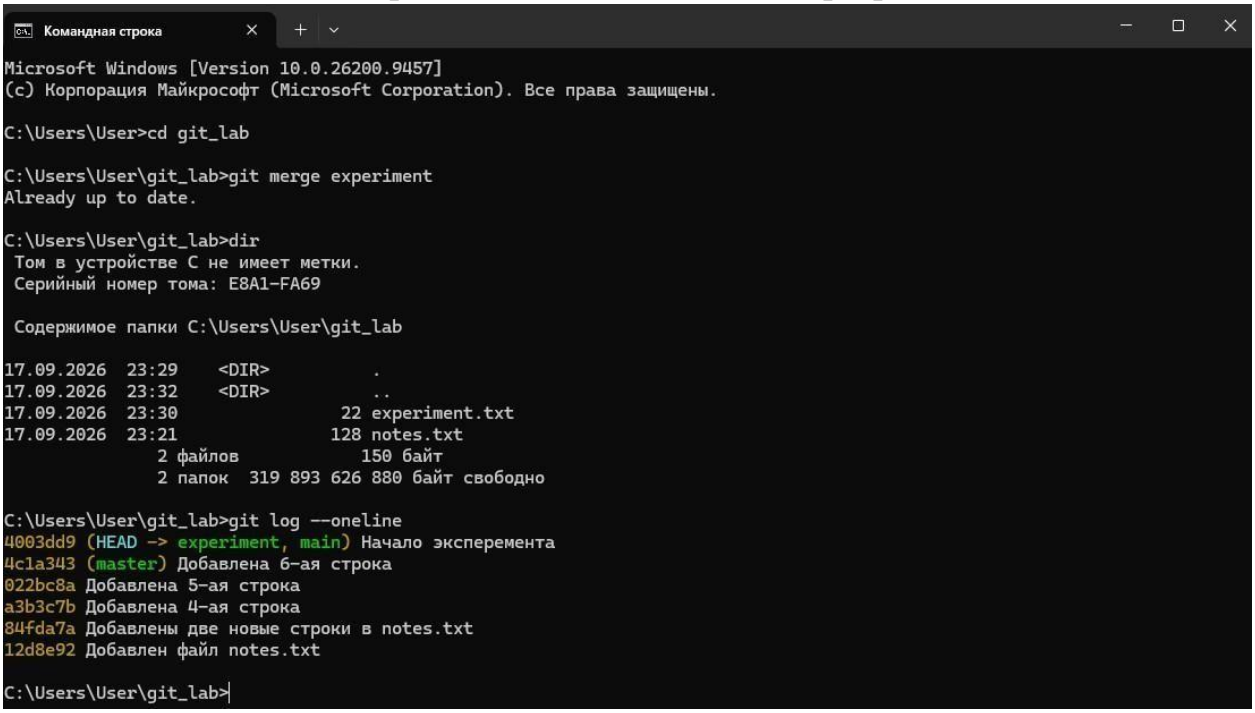



Рисунок 6. Слияние веток

## Задание 7: Отмена незакоммиченного изменения

Вносим временное изменение в notes.txt и смотрим разницу через git diff. Если правка не нужна, отменяем её командой git restore — она возвращает файл к состоянию последнего коммита. Проверяем содержимое файла через type notes.txt: видим, что лишние строки исчезли.

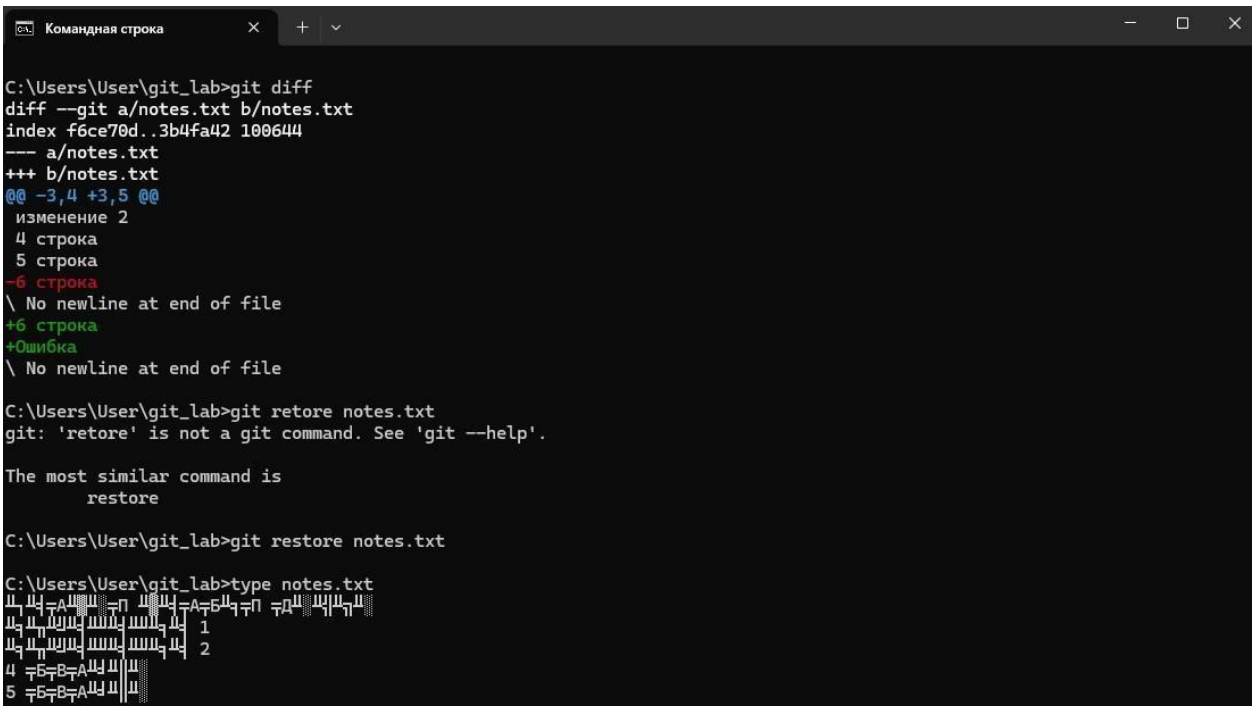



Рисунок 7. Отмена изменения(незакоммиченного)

## Задание 8: Работа с индексом

Вносим изменение в файл и добавляем его в индекс через git add. Смотрим статус: Git показывает, что файл подготовлен к коммиту. Затем убираем изменение из индекса командой git restore --staged, но оставляем правки в самом файле. Снова проверяем статус: теперь Git снова видит изменения как «не подготовленные к коммиту». Так мы разделяем состояние файла на диске и то, что Git планирует закоммитить. Изменение осталось в файле, но убрано из индекса

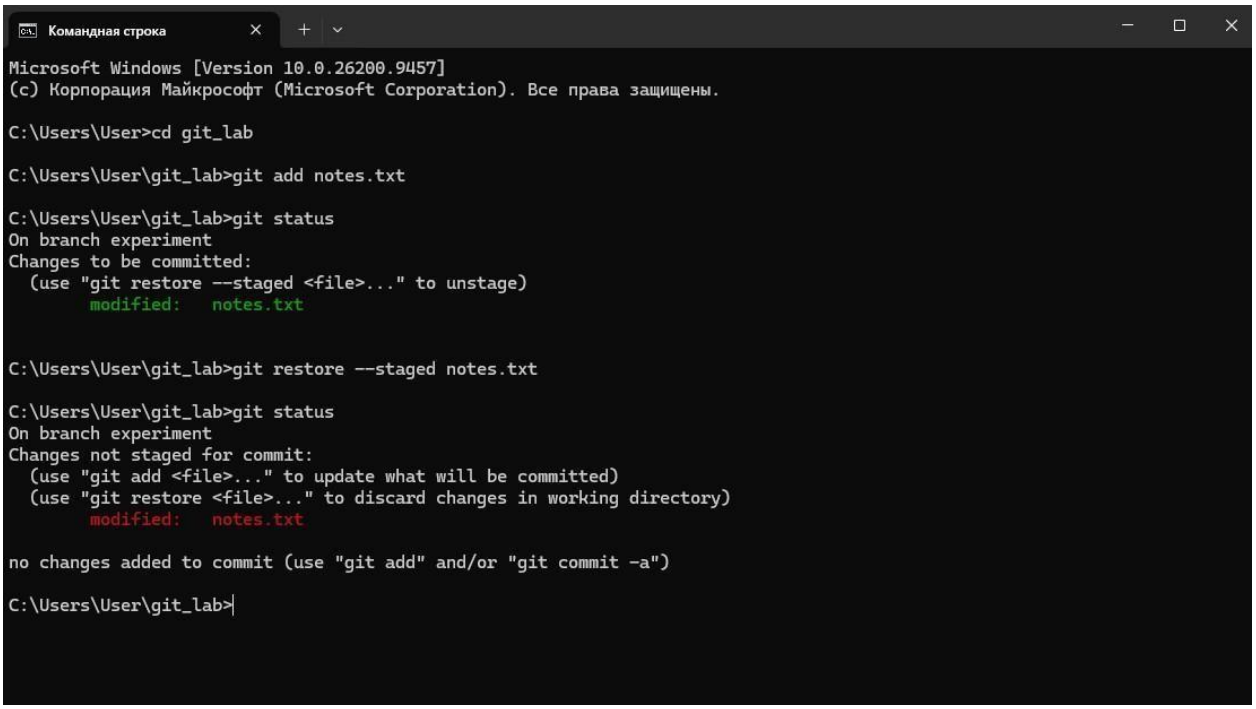



Рисунок 8. Работа с индексом

## Задание 9: Отмена коммита

Вносим тестовое изменение, добавляем и коммитим его. Смотрим историю через git log --oneline, копируем хеш коммита (первые 7 символов). Выполняем git revert с этим хешем — Git создаёт новый коммит-отмену, который компенсирует изменения. Проверяем историю: исходный коммит остаётся, а рядом появляется коммит отмены. Так Git сохраняет полную историю без переписывания прошлого**.** В итоге отмененный коммит не исчезнет.

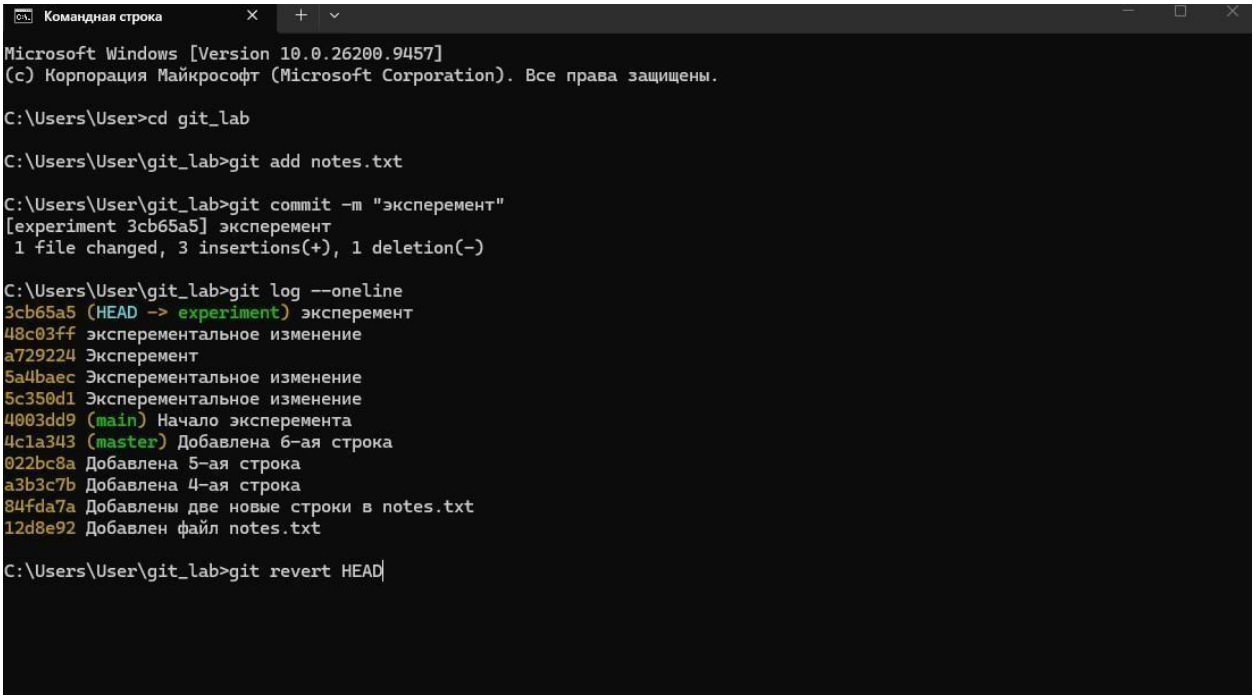



Рисунок 9. Отмена коммита

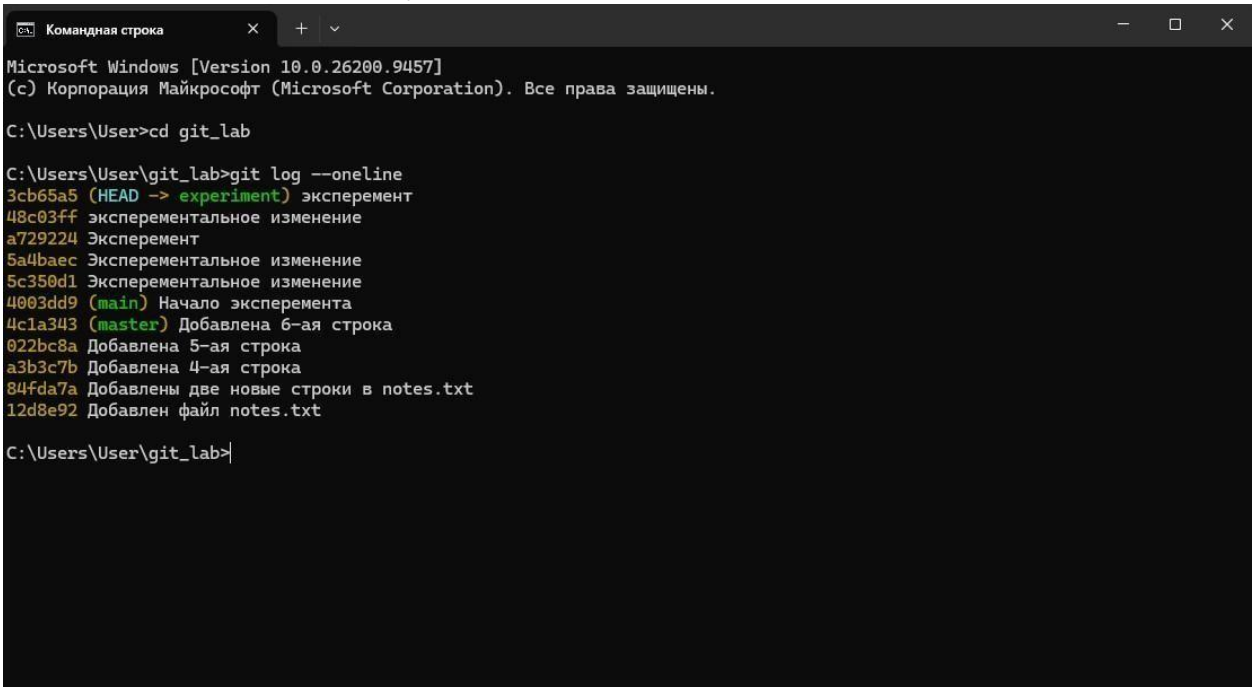



Рисунок 10. Результат отмены

## Задание 10: Самостоятельная работа

1). Создаём свою ветку project и переключаемся на неё. Создаём и изменяем файл test.txt, делаем два коммита

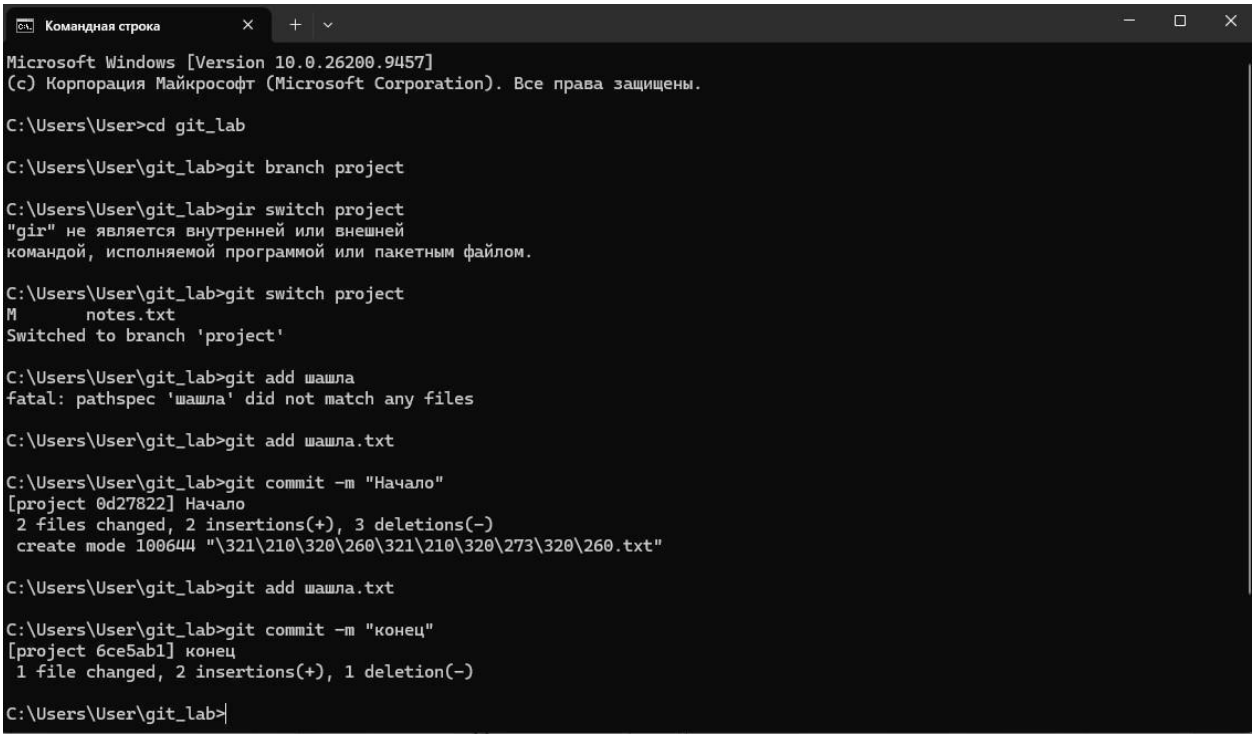



Рисунок 11. Первая часть задания

2). Смотрим разницу между этими коммитами через git diff HEAD~1..HEAD — так видим, какие именно строки изменились.

Возвращаемся в main и сливаем ветку. Проверяем итоговую историю: все коммиты и коммит слияния на месте, файл шашла.txt теперь есть в main.

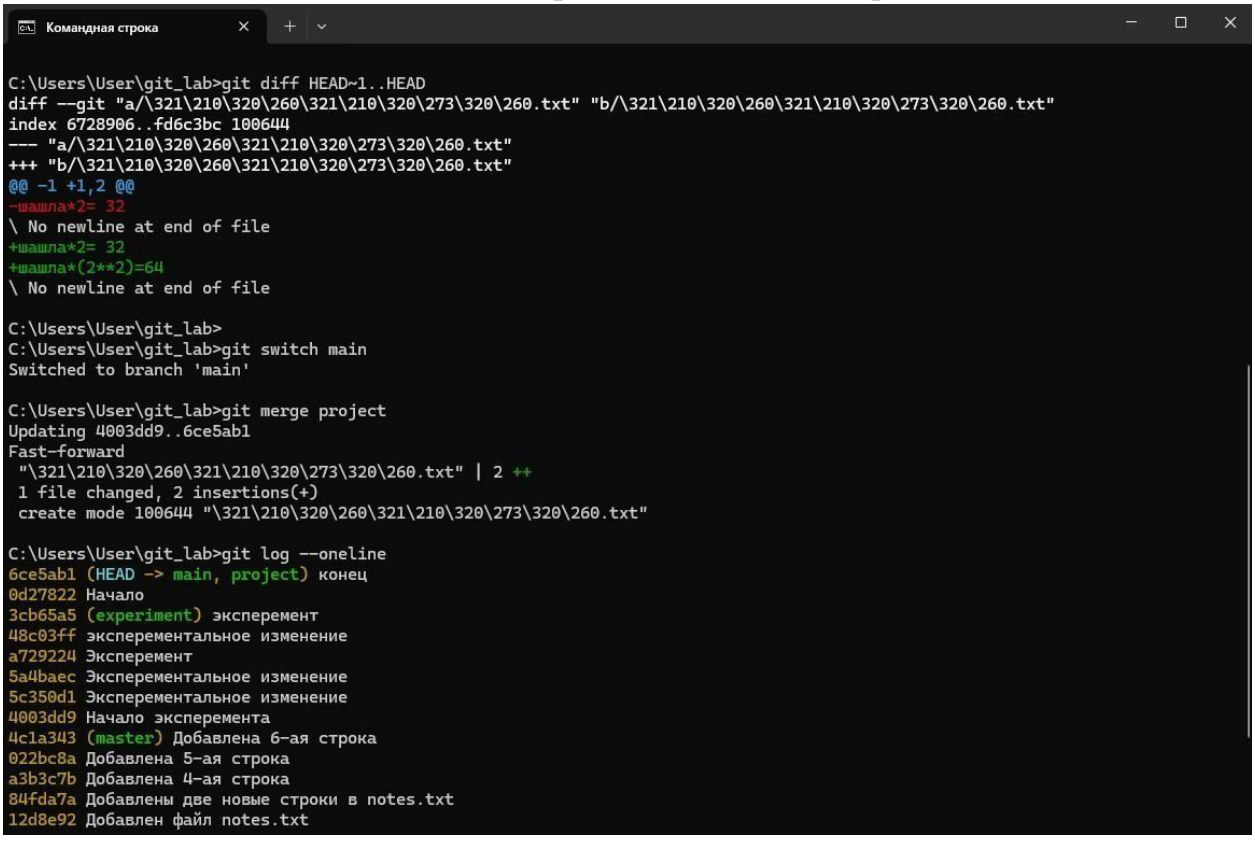


Рисунок 12. Вторая часть задания

**ВЫВОД:**

В ходе лабораторной работы были освоены базовые операции Git: инициализация репозитория, создание коммитов, работа с ветками и слиянием, просмотр истории и различий, отмена изменений и коммитов.